Chapter 1

System Development and Experimental Evaluation

1.1 Acoustic Representations and Learning Techniques

A key design decision in the project was to use different acoustic representations and learning strategies for different modules, rather than a single uniform feature extraction pipeline. This decision follows directly from the conceptual framework: different Tajweed rule categories have different acoustic signatures and require different forms of evidence.

1.1.1 MFCC Features for Rule-Oriented Modules

The duration, transition, and burst modules all use mel-frequency cepstral coefficient (MFCC) features as their primary acoustic representation. The MFCC pipeline extracts base coefficients together with first- and second-order delta coefficients, which are concatenated to produce a richer frame-level vector.

This choice was motivated by three properties of the target task. First, duration and burst judgments are highly local and temporal: whether a sound was held long enough or whether a consonantal release occurred are questions that can be answered from a short local window of frames. Second, MFCC features are computationally efficient and have a well-established track record in phoneme-level and rule-level speech classification tasks. Third, the rule-module inputs are relatively short audio segments rather than full-verse sequences, so the long-range context modelling that benefits end-to-end ASR systems is less critical here.

1.1.2 Self-Supervised Features for Content Verification

Content verification is structurally different from Tajweed rule classification: it requires recovering the recited text from audio, a task much closer to automatic speech recognition. For this reason, the content path used self-supervised learning (SSL) representations obtained through a wav2vec-style feature extractor.

When the pretrained `torchaudio` wav2vec bundle was available, the content model used those speech representations as its acoustic frontend. A deterministic fallback extractor was also implemented so that the pipeline remained runnable in environments where the pretrained bundle was unavailable. The rationale for using SSL features in the content path is that they encode phoneme-like acoustic information at a higher level of abstraction than hand-crafted features, which makes them better suited to sequence-level text recovery.

At a later stage of development, the SSL-plus-CTC content path was replaced by a fine-tuned Whisper model (see Section **??**). In the final system, Whisper serves as the acoustic and language model for content verification, and the SSL/CTC path is retained only as a documentation of the development history.

1.1.3 CTC-Based Training and Decoding

The original content model was trained with a connectionist temporal classification (CTC) objective. CTC predicts a frame-level probability distribution over characters and a blank symbol; a decoder then converts these distributions into a final text hypothesis.

Several decoder variants were evaluated:

- **Greedy decoding**: selects the most probable character at each frame and collapses repeats.
- **Prefix beam search**: maintains a beam of partial hypotheses and is generally more accurate than greedy decoding.
- **Blank-penalty tuning**: reduces the model's tendency to emit excessive blank tokens, which was a persistent problem in early content experiments.
- **Lexicon-constrained decoding**: restricts the output to a canonical vocabulary of known Qur'anic chunks. This produced the strongest CTC-era content results.

1.1.4 Whisper Fine-Tuning

The final content gate uses a fine-tuned Whisper model rather than a CTC model. Whisper is an encoder–decoder ASR system capable of open-ended sequence generation, which makes it better suited to full-ayah content verification than the CTC path.

Fine-tuning proceeded in two passes. The first pass (v1) trained Whisper-medium on a clean baseline manifest, excluding a problematic reciter subset that dominated errors and introduced instability. The second pass (v2) continued training from the v1 checkpoint, applying targeted oversampling of difficult reciters and longer ayah-length buckets. Training used gradient accumulation, mixed-precision computation, and gradient checkpointing throughout.

1.1.5 Experimental Methodology: Promotion Gates

The project used an explicit promotion-gate methodology: no component was adopted as the official baseline until it had been compared against the previous baseline on a consistent held-out evaluation set. Candidate improvements that degraded overall performance were explicitly rejected even when they improved one sub-metric.

Notable rejected candidates include:

- chunked-content hard-case retraining (improved ghunnah recall but degraded madd accuracy, worsening the overall result);
- a larger chunked content model on a stricter text-held-out split (did not consistently outperform the promoted baseline);
- raw beam-search decoding for the CTC content model (performed below the lexicon-constrained variant).

This gate-based approach means the final system is not simply the most complex version. It is the version that consistently survived side-by-side evaluation.

1.2 Rule Metadata Manifest for Pedagogical Feedback

The proposed system does not stop at detecting whether a recitation segment is correct or incorrect. Its objective is to transform technical diagnostic outputs into meaningful pedagogical feedback. For this reason, an additional

structured resource was introduced: the *rule metadata manifest*. This manifest is a machine-readable JSON file that stores pedagogical, technical, and severity-related metadata for each Tajweed rule handled by the system.

The manifest is not responsible for detecting errors directly. Error detection is performed by the specialised modules of the architecture. The role of the manifest is to interpret the structured diagnosis produced by these modules and to provide the feedback generation layer with the information required to explain the error to the learner. In this sense, the manifest acts as a pedagogical knowledge layer between the diagnosis aggregation stage and the final learner-facing feedback.

1.2.1 Motivation

The need for the rule metadata manifest comes from the difference between a technical diagnosis and a useful pedagogical explanation. A model output such as `expected_rule = ghunnah`, `source_module = duration`, and `diagnosis_label = insufficient_ghunnah_duration` is meaningful for the system, but it is not sufficient as learner feedback. The learner needs to know which rule was involved, what was expected, what went wrong, and how the error can be corrected.

Without a manifest, the feedback generation layer would have to infer this information directly from technical labels. This would make the feedback less controlled, less consistent, and more difficult to extend. The manifest solves this problem by explicitly storing, for each rule, its pedagogical description, expected behaviour, common error types, default error messages, corrective messages, positive messages, and feedback priority.

The manifest also supports the central design principle of the architecture: feedback must be generated from a structured diagnosis report rather than from raw audio. The audio signal is analysed by the appropriate specialist module, the aggregation layer organises the resulting evidence, and the manifest translates that diagnosis into rule-aware pedagogical language.

1.2.2 Position in the System Architecture

The rule metadata manifest is used after diagnosis aggregation layer combines content judgments, rule judgments, confidence values, and diagnostic evidence into a structured diagnosis report. The feedback generation layer uses the diagnosis report together with the rule metadata manifest to produce learner-facing feedback.

Thus, the annotated Qur'anic reference and the rule manifest have complementary roles. The annotated reference indicates *where* each rule applies

in the canonical text. The manifest explains *what* each rule means, *how* it should be realised, *what* types of errors may occur, and *how* those errors should be presented to the learner.

1.2.3 Main Components of the Manifest

Each rule entry in the manifest follows a common structure. The first group of fields identifies the rule:

- `rule_id`: the unique technical identifier used by the system;
- `arabic_name`: the formal Arabic name of the rule;
- `english_name`: the English name of the rule;
- `display_name`: the bilingual learner-facing name.

A second group of fields links the rule to the modular architecture:

- `family`: the Tajweed family to which the rule belongs;
- `acoustic_category`: the acoustic class of the rule, such as `duration`, `transition`, `burst`, or `contextual`;
- `source_module`: the specialist module responsible for analysing this rule;
- `status`: whether the rule is currently active in the implemented system or planned for future extension.

A third group stores pedagogical information:

- `pedagogical_description`: a simple explanation of the rule;
- `expected_behavior`: a description of correct realisation;
- `acoustic_cues`: technical cues relevant to the model;
- `learner_cues`: simple instructions that can be shown to the learner.

Finally, each rule contains feedback-related fields:

- `error_types`: possible errors associated with the rule;
- `default_error_message`: a message describing what went wrong;

- `corrective_message`: a message explaining how to correct the error;
- `generic_error`: a fallback message when the exact error type is uncertain;
- `positive_message`: a reinforcement message when the rule is correctly realised.

For example, a simplified entry for the rule of Ghunnah may be represented as Listing **??**.

```
{
  "rule_id": "ghunnah",
  "arabic_name": "الغنة",
  "english_name": "Nasalization",
  "acoustic_category": "duration",
  "source_module": "duration",
  "classical_severity": "lahn_khafi",
  "severity_level": "high",
  "expected_behavior": {
    "summary": {
      "en": "Produce a clear nasal sound and hold it for
         two counts.",

    }
  },
  "error_types": {
    "too_short": {
      "diagnosis_label": "insufficient_ghunnah_duration",
      "default_error_message": {
        "en": "The ghunnah was too short.",
         },
      "corrective_message": {
        "en": "Hold the nasal sound for two counts.",

      }
    }
  }
}
```

This structure allows the feedback layer to move from a technical diagnosis to a controlled pedagogical message without hardcoding the explanation inside the model or the aggregation logic.

1.2.4 Severity and Feedback Priority

A major extension of the manifest is the inclusion of feedback priority and error severity. This is necessary because a learner may produce several errors in the same recitation. The system must therefore decide which errors should be shown first, which should be grouped, and which should be delayed or treated as minor refinements.

The severity model follows the classical distinction between *Laḥn Jalī* and *Laḥn Khafī*. In system terms, *Laḥn Jalī* corresponds mainly to content-level errors. These include substitution, deletion, insertion, and vowel distortion when they affect the identity of the recited unit. Such errors are treated as critical because they affect what was recited.

By contrast, *Laḥn Khafī* corresponds mainly to rule-level Tajweed errors. In this case, the intended word or segment is still identifiable, but the expected manner of recitation was not correctly realised. Examples include a Madd that is too short, a weak or missing Ghunnah, an incorrectly realised Ikhfā', or a missing Qalqalah release.

The manifest therefore assigns each rule or error type a set of severity fields:

- `classical_severity`: the classical category, such as `lahn_jali` or `lahn_khafi`;
- `severity_level`: the practical feedback level, such as `critical`, `high`, `medium`, or `low`;
- `base_score`: a numeric score used for ordering feedback;
- `feedback_priority`: the display priority used by the feedback generation layer.

The default ordering policy is:

1. content-level errors are shown before rule-level errors;
2. higher severity scores are shown first;
3. if severity is similar, higher-confidence errors are shown first;
4. if both severity and confidence are similar, the Qur'anic order of the verse is preserved.

This policy prevents the system from overwhelming the learner with an unordered list of messages. It also prevents misleading feedback. For example, if a learner omits the segment that should contain a Madd, the system should not first say that the Madd was too short. The more accurate feedback is that the expected content was not recited. Only after the content is present does rule-level feedback become meaningful.

1.2.5 Role in the Feedback Generation Layer

The rule metadata manifest is used after the aggregation layer has produced the structured diagnosis report. At this stage, the system already knows the main diagnostic information for each affected position, including the `position`, `canonical_unit`, `aligned_interval`, `content_judgment`, `expected_rule`, `source_module`, `rule_judgment`, `evidence`, `diagnosis_label`, and `feedback_priority`. The manifest does not create these fields; it uses them to transform the technical diagnosis into learner-facing feedback.

The feedback layer first checks the `content_judgment`. If the entry contains a content-level error such as deletion, insertion, or substitution, the system uses the content feedback templates from the manifest and does not prioritise rule feedback at the same position. This preserves the content-first logic of the architecture: the learner must first correct what was recited before receiving detailed feedback about how the Tajweed rule was performed.

If the content is correct but the `rule_judgment` is incorrect, the feedback layer uses `expected_rule` to retrieve the corresponding rule entry from the manifest. It then uses `source_module`, `diagnosis_label`, and the available `evidence` to select the most appropriate error type. For example, a diagnosis entry with `expected_rule = ghunnah`, `source_module = duration`, and evidence indicating that the duration was shorter than expected is mapped to the Ghunnah error type `too_short`. The manifest then provides the bilingual default error message and corrective message for that case.

In this way, the manifest acts as a controlled lookup layer between diagnosis and feedback. The diagnosis report determines what happened, where it happened, and which module detected it. The manifest determines how that diagnosis should be explained, translated, and prioritised for the learner.

1.2.6 Contribution to the System

The rule metadata manifest contributes to the system in several ways. First, it makes the feedback rule-specific. Instead of returning a generic incorrect label, the system can explain that the Ghunnah was too short, that the Ikhfā' was too clear, or that the Qalqalah release was missing.

Second, it makes the feedback pedagogical. Each message includes not only what went wrong, but also how the learner can correct the error.

Third, it makes the feedback consistent. Since the messages are stored in a central manifest, the feedback generator does not need to invent explanations for each diagnosis.

Fourth, it supports prioritisation. When several errors occur in one recitation, the system can organise them according to content priority, severity, confidence, and Qur'anic order.

Finally, it supports extensibility. New rules can be added by creating new entries in the manifest. Existing modules, routing logic, aggregation logic, and feedback templates can remain unchanged as long as the new rule follows the same metadata structure.

1.2.7 Conclusion

The rule metadata manifest is a key component in moving the system from automatic Tajweed error detection toward automatic Tajweed instruction. The detection modules identify acoustic and content-level deviations, and the aggregation layer organises them into a structured diagnosis. The manifest then supplies the pedagogical knowledge required to explain these deviations in a clear, bilingual, prioritised, and learner-oriented form.

Therefore, the manifest should not be viewed as a simple configuration file. It is a pedagogical knowledge layer that connects rule-aware diagnosis with meaningful feedback. Its presence allows the system to provide not only a technical judgment, but also an educational response that helps the learner understand and correct the recitation error.